



**CHENNAI
INSTITUTE OF TECHNOLOGY**
(Autonomous)



Accredited by
NBA
CSE, ECE, EEE, MECH, MCT

nirf
175th Rank
(NIRF Ranking 2022)

CS4304 ARTIFICIAL INTELLIGENCE
[R2024]
LECTURE NOTES

B.TECH II YEAR – III SEM (R24)
(2025-2026)

CHENNAI INSTITUTE OF TECHNOLOGY

(Autonomous Institution – UGC, Govt. of India)

Recognized under 2(f) and 12 (B) of UGC ACT 1956

Sarathi Nagar, Kundrathur, Chennai – 600069

Course Code	ARTIFICIAL INTELLIGENCE	L	T	P	C
CS4304		3	0	2	4

COURSE OBJECTIVES:

The main objectives of this course are to:

- Learn the fundamental AI approaches.
- Develop problem-solving skills relevant to AI.
- Explore Python and its libraries used in Artificial Intelligence.
- Grasp logical reasoning and apply it in decision-making.
- Understand the concept of probabilistic reasoning in real-time scenarios.

Course Description

This introductory course provides students with a foundational understanding of Artificial Intelligence (AI).

Prerequisites

- Basic programming skills (preferably in Python).
- Familiarity with fundamental concepts in mathematics and computer science.

UNIT I: Introduction to Artificial Intelligence 9

Introduction: Foundations of AI – Intelligent agents: Environments, structure of agents – Problem-solving: Searching for solutions, uninformed and informed search strategies, heuristic functions.

UNIT II: Advanced Search Strategies 9

Local Search Algorithms and Optimization Problems – Local search in Continuous Spaces –Global optimization algorithms: Genetic Algorithms, Particle Swarm Optimization, Ant Colony Optimization, Constraint Satisfaction Problems (CSP): Backtracking and local search for CSPs – Adversarial Search: Games, Optimal decisions, alpha-beta pruning, imperfect real-time decisions, stochastic games, partially observable games.

UNIT III: Knowledge Representation 9

Logical systems and Knowledge-Based systems – Representing knowledge using Propositional Logic – Inference in Propositional Logic: Laws of Inference, Forward Chaining, Backward Chaining, Resolution – Representing knowledge using First Order Logic –Inference in First Order Logic: Unification, Forward Chaining, Backward Chaining.

UNIT IV: Planning and Quantifying Uncertainty 9

Classical Planning: Definition, Algorithms for Planning as State-Space Search, Planning Graphs – Classical Planning Approaches – Planning and Acting: Time, Schedules, and Resources, Hierarchical Planning – Planning and Acting in Nondeterministic Domains, Multi-agent Planning – Acting under Uncertainty: Conditional Independence, Bayes Rule, Naive Bayes Classifier, Bayesian Belief Network – Inference in Bayesian Belief Network – Making Decisions in Complex Environments: Markov Decision Processes.

UNIT V: Reinforcement Learning and AI Applications 9

Introduction to Reinforcement Learning: Bandit algorithms, Policy Gradient, Markov Decision Processes (MDPs) – Dynamic Programming & Temporal Difference (TD) Methods – Markov Chain Monte Carlo (MCMC) Methods – Future Trends in AI: Quantum AI, Human-AI Collaboration, Explainable AI, AI in Edge Computing, Artificial General Intelligence – Overview of AI applications in various fields: Natural Language Processing (NLP), Computer Vision, Robotics, Business Intelligence, Healthcare, Education, Finance, Manufacturing.

Course Format

Lectures and discussions, Hands-on coding exercises and projects, Online resources and tutorials.

Practical Exercises

Suggested exercises include implementing search strategies, solving constraint satisfaction problems, developing game AI, and experimenting with basic reinforcement learning algorithms using Python.

Assessments & Grading

Quizzes / Assignments, Project, Three Internal Assessments, Final Examination

COURSE OUTCOMES:

Upon completion of the course, the students will be able to:

CO1: Identify and apply agent-based engineering solutions using AI tools.

CO2: Demonstrate knowledge of advanced search strategies.

CO3: Apply knowledge representation, reasoning, and planning in AI solutions.

CO4: Establish semantic relationships between events and actions in decision-making.

CO5: Solve complex decision-making problems using Markovian properties.

TEXT BOOKS:

4. Russel, S. and Norvig, P., *Artificial Intelligence – A Modern Approach*, 4th Edition, Pearson Education, 2021.
5. Sutton, R. S., & Barto, A. G., *Reinforcement Learning: An Introduction*, 2nd Edition, MIT Press, 2020.

REFERENCES:

9. Vinod Chandra. S.S, and Anand Hareendran, S., *Artificial Intelligence and Machine Learning*, PHI Publishers, 2014.
10. Luger, G., *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th Edition, Pearson Education, 2008.
11. Nilsson, J., *Artificial Intelligence: A New Synthesis*, Elsevier Publishers, 1998.

YouTube Resources:

1. **3Blue1Brown**: Mathematical foundations of AI algorithms.
2. **Sentdex**: Python programming tutorials for AI and machine learning.
3. **Artificial Intelligence - All in One**: Comprehensive tutorials on various AI topics.
4. **Andrew Ng**: Lectures on AI and machine learning concepts.
5. **Google Developers**: Tutorials and demos on AI tools and technologies.

TOTAL: 75 PERIODS

UNIT I: Introduction to Artificial Intelligence

9

Introduction: Foundations of AI – Intelligent agents: Environments, structure of agents – Problem-solving: Searching for solutions, uninformed and informed search strategies, heuristic functions.

Introduction:

- Artificial Intelligence is concerned with the design of intelligence in an artificial device. The term was coined by John McCarthy in 1956.
- Intelligence is the ability to acquire, understand, and apply knowledge to achieve goals in the world.
- AI is the study of the mental faculties through the use of computational models
- AI is the study of intellectual/mental processes as computational processes.
- AI programs will demonstrate a high level of intelligence to a degree that equals or exceeds the intelligence required of a human in performing some task.
- AI is unique, sharing borders with Mathematics, Computer Science, Philosophy, Psychology, Biology, Cognitive Science and many others.
- Although there is no clear definition of AI or even Intelligence, it can be described as an attempt to build machines that like humans can think and act, able to learn and use knowledge to solve problems on their own.

Sub Areas of AI:

1) Game Playing

Deep Blue Chess program beat world champion Gary Kasparov

2) Speech Recognition

PEGASUS spoken language interface to American Airlines' EASY SABRE reservation system, which allows users to obtain flight information and make reservations over the

telephone. The 1990s has seen significant advances in speech recognition so that limited systems are now successful.

3) Computer Vision

Face recognition programs in use by banks, government, etc. The ALVINN system from CMU autonomously drove a van from Washington, D.C. to San Diego (all but 52 of 2,849 miles), averaging 63 mph day and night, and in all weather conditions. Handwriting recognition, electronics and manufacturing inspection, photo interpretation, baggage inspection, reverse engineering to automatically construct a 3D geometric model.

4. Expert Systems

Application-specific systems that rely on obtaining the knowledge of human experts in an area and programming that knowledge into a system.

- a. **Diagnostic Systems:** MYCIN system for diagnosing bacterial infections of the blood and suggesting treatments. Intellipath pathology diagnosis system (AMA approved). Pathfinder medical diagnosis system, which suggests tests and makes diagnoses. Whirlpool customer assistance center.

5. System Configuration

DEC's XCON system for custom hardware configuration. Radiotherapy treatment planning.

6. Financial Decision Making

Credit card companies, mortgage companies, banks, and the U.S. government employ AI systems to detect fraud and expedite financial transactions. For example, AMEX credit check.

7. Classification Systems

Put information into one of a fixed set of categories using several sources of information. E.g., financial decision making systems. NASA developed a system for classifying very faint areas in astronomical images into either stars or galaxies with very high accuracy by learning from human experts' classifications.

8. Mathematical Theorem Proving

Use inference methods to prove new theorems.

9. Natural Language Understanding

AltaVista's translation of web pages. Translation of Caterpillar Truck manuals into 20 languages

10. Scheduling and Planning

Automatic scheduling for manufacturing. DARPA's DART system used in Desert Storm and Desert Shield operations to plan logistics of people and supplies. American Airlines rerouting contingency planner. European space agency planning and scheduling of spacecraft assembly, integration and verification.

11. Artificial Neural Networks:

12. Machine Learning

Applications of AI:

AI algorithms have attracted the close attention of researchers and have also been applied successfully to solve problems in engineering. Nevertheless, for large and complex problems, AI algorithms consume considerable computation time due to the stochastic feature of the search approaches

1. Business; financial strategies
2. Engineering: check design, offer suggestions to create new products, and expert systems for all engineering problems
3. Manufacturing: assembly, inspection and maintenance
4. Medicine: monitoring, diagnosing

5. Education: in teaching
6. Fraud detection
7. Object identification
8. Information retrieval
9. Space shuttle scheduling

Building AI Systems:

1) Perception

Intelligent biological systems are physically embodied in the world and experience the world through their sensors (senses). For an autonomous vehicle, input might be images from a camera and range information from a rangefinder. For a medical diagnosis system, perception is the set of symptoms and test results that have been obtained and input to the system manually.

2) Reasoning

Inference, decision-making, classification from what is sensed and what the internal "model" is of the world. Might be a neural network, logical deduction system, Hidden Markov Model induction, heuristic searching a problem space, Bayes Network inference, genetic algorithms, etc.

Includes areas of knowledge representation, problem-solving, decision theory, planning, game theory, machine learning, uncertainty reasoning, etc.

3) Action

Biological systems interact within their environment by actuation, speech, etc. All behavior is centered around actions in the world. Examples include controlling the steering of a Mars rover or autonomous vehicle, or suggesting tests and making diagnoses for a medical diagnosis system. Includes areas of robot actuation, natural language generation, and speech synthesis.

The definitions of AI:

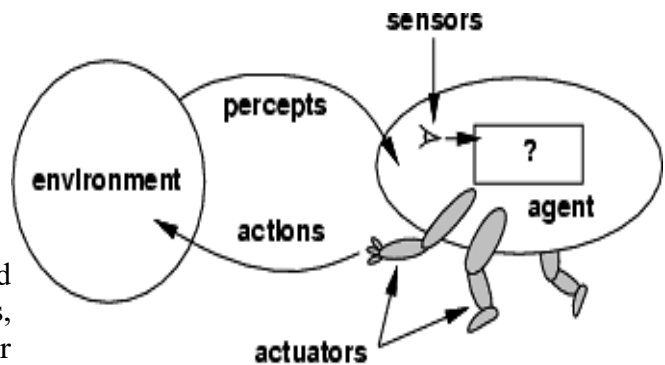
"The exciting new effort to make computers think . . . <i>machines with minds</i>, in the full and literal sense" (Haugeland, 1985) "The automation of activities that we associate with human thinking, activities such as decision-making, problem-solving, learning..." (Bellman, 1978)	"The study of mental faculties through the use of computational models" (Charniak and McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act" (Winston, 1992)
"The art of creating machines that perform functions that require intelligence when performed by people" (Kurzweil, 1990) "The study of how to make computers do things at which, at the moment, people are better" (Rich and Knight, 1991)	"A field of study that seeks to explain and emulate intelligent behavior in terms of computational processes" (Schalkoff, 1990) "The branch of computer science that is concerned with the automation of intelligent behavior" (Luger and Stubblefield, 1993)

Agents and Environments:

Agent:

An *Agent* is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.

- ✓ A *human agent* has eyes, ears, and other organs for sensors and hands, legs, mouth, and other body parts for actuators.
- ✓ A *robotic agent* might have cameras and infrared range finders for sensors and various motors for actuators.
- ✓ A *software agent* receives keystrokes, file contents, and network packets as sensory inputs and acts on the environment by displaying on the screen, writing files, and sending network packets.



Percept:

We use the term percept to refer to the agent's perceptual inputs at any given instant.

Percept Sequence:

An agent's percept sequence is the complete history of everything the agent has ever perceived.

Agent function:

Mathematically speaking, we say that an agent's behavior is described by the agent function that maps any **given percept sequence to an action**.

Agent Program:

Internally, the agent function for an artificial agent will be implemented by an agent program. It is important to keep these two ideas distinct. The agent function is an abstract mathematical description; the agent program is a concrete implementation, running on the agent architecture.

To illustrate these ideas, we will use a very simple example - the a vacuum-cleaner world shown in **Fig 2.1.5**. This particular world has just two locations: squares A and B. The vacuum agent perceives which square it is in and whether there is dirt in the square. It can choose to move left, move right, suck up the dirt, or do nothing. One very simple agent function is the following: if the current square is dirty, then suck, otherwise move to the other square. A partial tabulation of this agent function is shown in **Fig 2.1.6**.

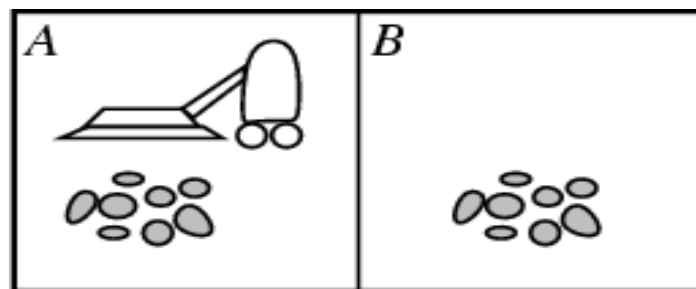


Fig 2.1.5: A vacuum-cleaner world with just two locations.

Agent function

Percept Sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
...	

Fig 2.1.6: Partial tabulation of a simple agent function for the example: vacuum-cleaner world shown in the Fig2.1.5

```
Function REFLEX-VACCUM-AGENT ([location, status]) returns an  
action  
If status=Dirty then return Suck  
else if location = A then return Right  
else if location = B then return Left
```

Fig 2.1.6(i): The REFLEX-VACCUM-AGENT program is invoked for each new percept (location, status) and returns an action each time

A **Rational agent** is one that does the right thing. we say that the right action is the one that will cause the agent to be most successful. That leaves us with the problem of deciding how and when to evaluate the agent's success.

We use the term performance measure for the how—the criteria that determine how successful an agent is.

- ✓ Ex-Agent cleaning the dirty floor
- ✓ Performance Measure-Amount of dirt collected
- ✓ When to measure-Weekly for better results

What is rational at any given time depends on four things:

- The performance measure defining the criterion of success
- The agent's prior knowledge of the environment
- The actions that the agent can perform
- The agent's percept sequence up to now.

Omniscience ,Learning and Autonomy:

- We need to distinguish between rationality and omniscience. An **Omniscient agent** knows the actual outcome of its actions and can act accordingly but omniscience is impossible in reality.

- Rational agent not only gathers information but also **learns** as much as possible from what it perceives.
- If an agent just relies on the prior knowledge of its designer rather than its own percepts then the agent lacks **autonomy**.
- A system is autonomous to the extent that its behavior is determined by its own experience.
- A rational agent should be autonomous.

E.g., a clock (lacks autonomy)

- No input (percepts)
- Run only by its own algorithm (prior knowledge)
- No learning, no experience, etc.

ENVIRONMENTS:

- ✓ The Performance measure, the environment, and the agents' actuators, and sensors come under the heading task environment. We also call this as **PEAS (Performance, Environment, Actuators, Sensors)**

Examples of Rational Agents

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	healthy patient, costs, lawsuits (court cases)	patient, hospital, stuff	display questions, tests, diagnoses, treatments,	keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	correct image categorization	downlink from orbiting satellite	display categorization of scene	color pixel arrays
Part-picking robot	percentage of parts in correct bins (box)	conveyor belt with parts, bins	jointed arm and hand	camera, joint angle sensors
Refinery controller معمل تكرير	purity, safety	Refinery, operators	valves pumps, heaters displays	temperature, pressure, chemical sensors
Interactive English tutor	student's score on test	set of students, testing agency	display exercises, corrections	keyboard entry
web crawling agent	did you get only pages you wanted	User, internet	Display related info	keyboard entry

Environment-Types:

1. Accessible vs. inaccessible or Fully observable vs Partially Observable:

If an agent's sensor can sense or access the complete state of an environment at each point of time then it is a fully observable environment, else it is partially observable.

2. Deterministic vs. Stochastic:

If the next state of the environment is completely determined by the current state and the actions selected by the agents, then we say the environment is deterministic.

3. Episodic vs. non-episodic:

- The agent's experience is divided into "episodes." Each episode consists of the agent perceiving and then acting. The quality of its action depends just on the episode itself because subsequent episodes do not depend on what actions occur in previous episodes.
- Episodic environments are much simpler because the agent does not need to think ahead.

4. Static vs. dynamic.

If the environment can change while an agent is deliberating, then we say the environment is dynamic for that agent; otherwise, it is static.

5. Discrete vs. continuous:

If there are a limited number of distinct, clearly defined percepts and actions we say that the environment is discrete. Otherwise, it is continuous.

6. Single-agent vs Multi-agents:

Single-agent: The AI system is the only entity taking action. Example- In traveling salesman problem.

Multi-agent: multiple agents are operating in the environment which can lead to competition, cooperation, or coexistence.

Ex. Poker or football games are multi-agent.

7. Known vs Unknown

Known: The outcomes of all actions are known in advance. Ex. The rules and outcomes of a board game are well.

Unknown: The outcomes of some actions are not known in advance and must be learned through interaction. Ex. All real-world scenarios.

We have to understand the nature of the environment and select the appropriate algorithm while designing the AI agent (or system).

Examples of task environments and their characteristics

- ✓ Crossword puzzle
- ✓ Chess with a clock
- ✓ Poker
- ✓ Backgammon
- ✓ Taxi driving
- ✓ Medical diagnosis and many more.

Task Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Cross word puzzle	Fully	Deterministic	sequential	Static	Discrete	Single
chess with a clock	Fully	Strategic	sequential	Static Semi	Discrete	multi
paper	partially	Strategic	sequential	Static	Discrete	multi
Backgammon	Fully	Stochastic	sequential	Static	Discrete	multi
Taxi driving	partially	Stochastic	sequential	Dynamic	Continuous	multi
medical diagnosis	partially	Stochastic	sequential	Dynamic	Continuous	single
Image Analysis	Fully	Deterministic	Episodic	Semi	Continuous	single
part-picking robot	partially	Stochastic	Episodic	Dynamic	Continuous	single
Refinery controller	partially	Stochastic	sequential	Dynamic	Continuous	single
Interactive English Tutor	partially	Stochastic	sequential	Dynamic	Discrete	multi

This Table shows 10 application and their task environment characteristics.

STRUCTURE OF INTELLIGENT AGENTS

- The job of AI is to design the agent program: a function that implements the agent mapping from percepts to actions. We assume this program will run on some sort of ARCHITECTURE computing device, which we will call the architecture.
- The architecture might be a plain computer, or it might include special-purpose hardware for certain tasks, such as processing camera images or filtering audio input. It might also include software that provides a degree of insulation between the raw computer and the agent program, so that we can program at a higher level. In general, the architecture makes the percepts from the sensors available to the program, runs the program, and feeds the program's action choices to the effectors as they are generated.
- The relationship among agents, architectures, and programs can be summed up as follows: agent = architecture + program

Agent Type	Percepts	Actions	Goals	Environment
Medical diagnosis system	Symptoms, findings, patient's answers	Questions, tests, treatments	Healthy patient, minimize costs	Patient, hospital
Satellite image analysis system	Pixels of varying intensity, color	Print a categorization of scene	Correct categorization	Images from orbiting satellite
Part-picking robot	Pixels of varying intensity	Pick up parts and sort into bins	Place parts in correct bins	Conveyor belt with parts
Refinery controller	Temperature, pressure readings	Open, close valves; adjust temperature	Maximize purity, yield, safety	Refinery
Interactive English tutor	Typed words	Print exercises, suggestions, corrections	Maximize student's score on test	Set of students

Figure 2.3 Examples of agent types and their PAGE descriptions.

Agent programs:

- Intelligent agents accept percepts from an environment and generates actions. The early versions of agent programs will have a very simple form (Figure 2.4)
- Each will use some internal data structures that will be updated as new percepts arrive.

These data structures are operated on by the agent's decision-making procedures to generate an action.

```

function TABLE-DRIVEN-AGENT(percept) returns action
  static: percepts, a sequence, initially empty
           table, a table, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(perceptstable)
  return action

```

Figure 2.5 An agent based on a prespecified lookup table. It keeps track of the percept sequence and just looks up the best action.

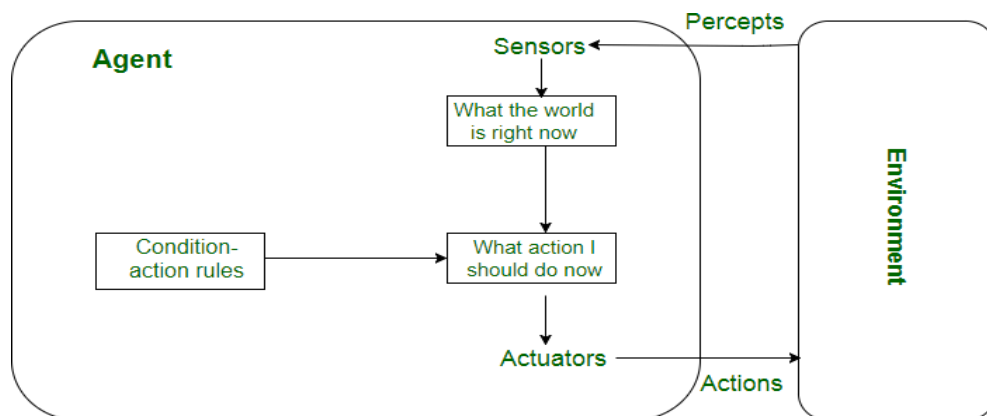
Types of agents:

Agents can be grouped into four classes based on their degree of perceived intelligence and capability:

- Simple Reflex Agents
- Model-Based Reflex Agents
- Goal-Based Agents
- Utility-Based Agents

Simple reflex agents:

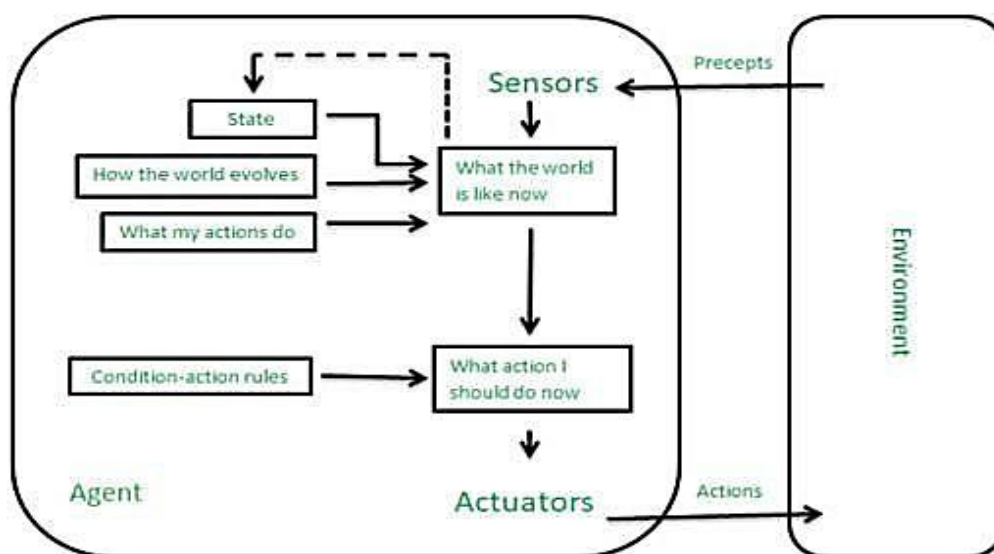
- Simple reflex agents ignore the rest of the percept history and act only on the basis of the current percept.
- The agent function is based on the condition-action rule.
- If the condition is true, then the action is taken, else not. This agent function only succeeds when the environment is fully observable.



Model-based reflex agents:

The Model-based agent can work in a partially observable environment and track the situation.

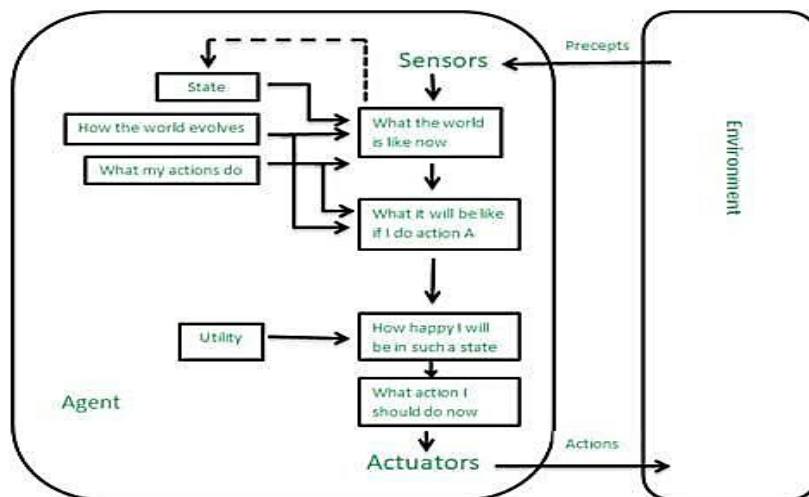
- A model-based agent has two important factors:
- **Model:** It is knowledge about "how things happen in the world," so it is called a Model-based agent.



- **Internal State:** It is a representation of the current state based on percept history.

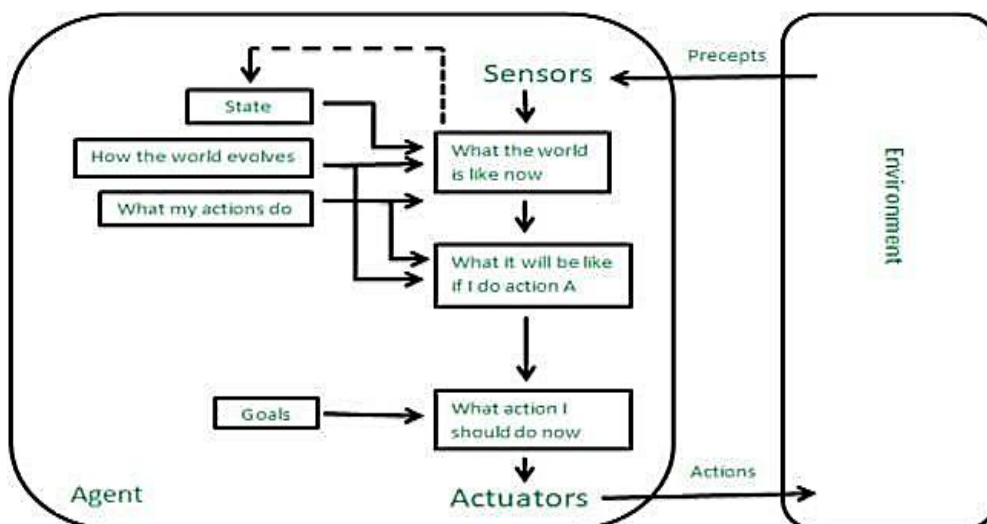
Goal-based agents:

- A goal-based agent has an agenda.
- It operates based on a goal in front of it and makes decisions based on how best to reach that goal.
- A goal-based agent operates as a search and planning function, meaning it targets the goal ahead and finds the right action in order to reach it.
- Expansion of the model-based agent.



Utility-based agents:

- Utility-based agent is an agent that acts based not only on what the goal is, but the best way to reach that goal.
- The Utility-based agent is useful when there are multiple possible alternatives, and an agent has to choose in order to perform the best action.
- The term utility can be used to describe how "happy" the agent is.



V) Learning Agents:

*A learning agent in AI is a type of agent that can learn from its past experiences or it has learning capabilities. It automatically adapts through learning.

A learning agent has the following components

Learning Element: It is responsible for making improvement by learning from the environment.

Critic: The learning element takes feedback from critic which describes how well the agent is doing with reference to a standard.

Performance element: It selects external action.

Problem generator: This suggests action which lead to new and informative experiences.

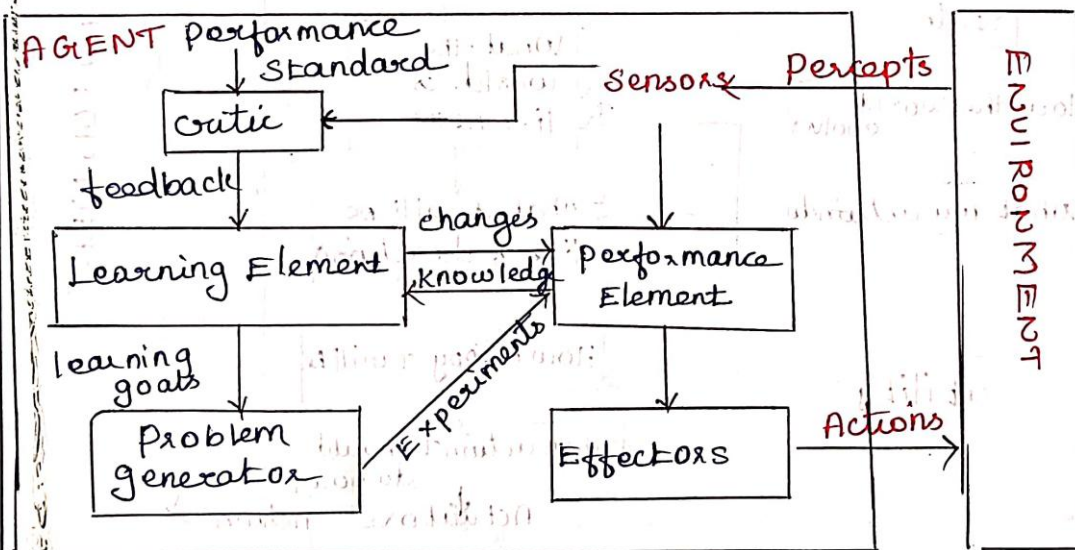


Fig shows a learning Agent.

Searching Techniques for AI

Problem-solving Agents – well-defined problems – uninformed search strategies – Informed search strategies.

Problem Solving Agents:

- Problem-solving agent is a goal-based agent.
- Problem-solving agents decide what to do by finding a sequence of actions that lead to desirable states.

Goal Formulation:

It organizes the steps required to formulate/ prepare one goal out of multiple goals available.

Problem Formulation:

It is a process of deciding what actions and states to consider to follow goal formulation. The process of looking for a best sequence to achieve a goal is called **Search**.

- A search algorithm takes a problem as input and returns a solution in the form of action sequences. Once the solution is found the action it recommends can be carried out. This is called **Execution phase**.

Well Defined problems and solutions:

A problem can be defined formally by 4 components:

The **initial state** of the agent is the state where the agent starts in. In this case, the initial state can be described as in: **Arad**

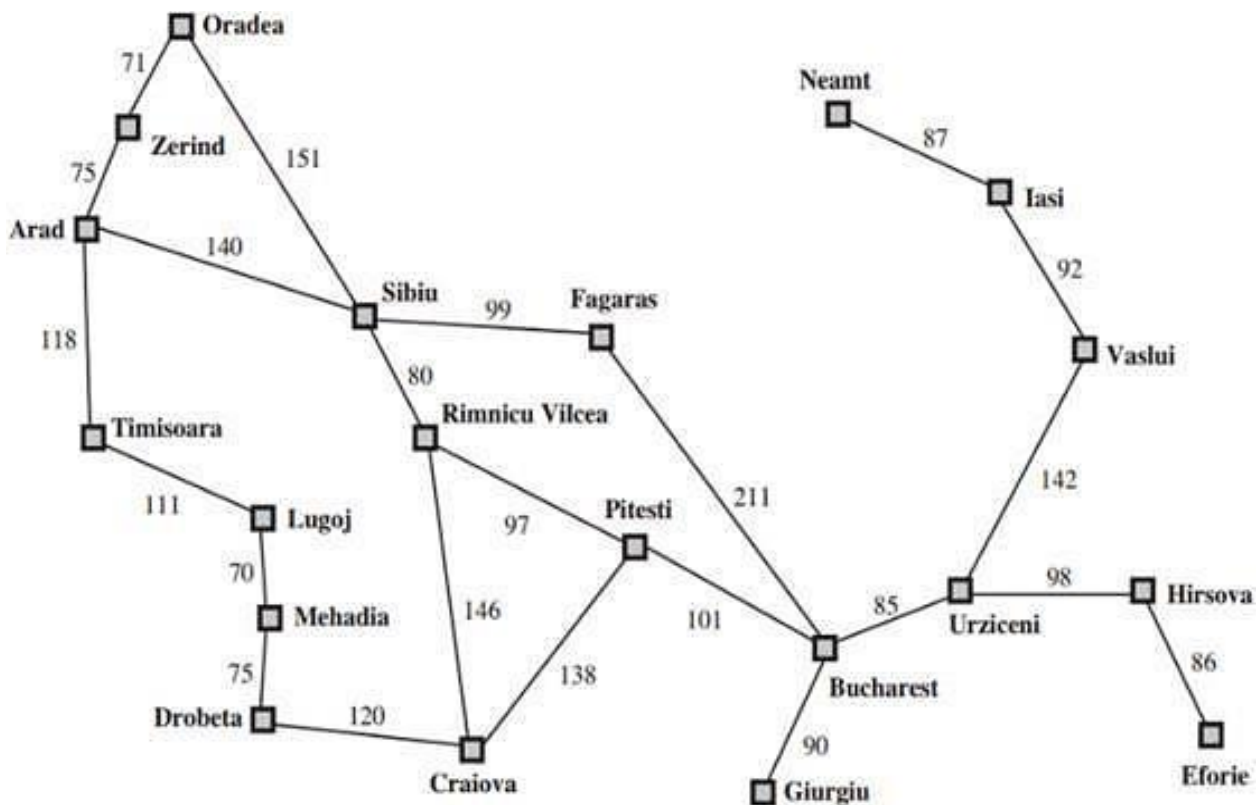
- The possible **actions** available to the agent, corresponding to each of the state the agent resides in.

For example, **ACTIONS(In: Arad) = {Go: Sibiu, Go: Timisoara, Go: Zerind}**.

Actions are also known as operations.

- A description of what each action does. The formal name for this is **Transition model**, specified by the function **Result(s,a)** that returns the state that results from the action **a** in state **s**.

We also use the term Successor to refer to any state reachable from a given state by a single action. For EX: **Result(In(Arad),GO(Zerind))=In(Zerind)**



Together the initial state, actions and transition model implicitly defines the **state space** of the problem. State space: set of all states reachable from the initial state by any sequence of actions

- **The goal test**, determining whether the current state is a goal state. Here, the goal state is {In: Bucharest}
- **The path cost function**, which determine the cost of each path, which is reflecting in the performance measure.

we define the cost function as $c(s, a, s')$, where s is the current state and a is the action performed by the agent to reach state s' .

```

function SIMPLE-PROBLEM-SOLVING-AGENT( $p$ ) returns an action
  inputs:  $p$ , a percept
  static:  $s$ , an action sequence, initially empty
            $state$ , some description of the current world state
            $g$ , a goal, initially null
            $problem$ , a problem formulation

   $state \leftarrow \text{UPDATE-STATE}(state, p)$ 
  if  $s$  is empty then
     $g \leftarrow \text{FORMULATE-GOAL}(state)$ 
     $problem \leftarrow \text{FORMULATE-PROBLEM}(state, g)$ 
     $s \leftarrow \text{SEARCH}(problem)$ 
   $action \leftarrow \text{RECOMMENDATION}(s, state)$ 
   $s \leftarrow \text{REMAINDER}(s, state)$ 
  return  $action$ 

```

Figure 3.1 A simple problem-solving agent.

Example:

8 puzzle problem

Initial State

2	8	3
1	6	4
7		5

Goal State

1	2	3
8		4
7	6	5

- **States:** a state description specifies the location of each of the eight tiles in one of the nine squares. For efficiency, it is useful to include the location of the blank.
- **Actions:** blank moves left, right, up, or down.
- **Transition Model:** Given a state and action, this returns the resulting state. For example if we apply left to the start state the resulting state has the 5 and the blank switched.
- **Goal test:** state matches the goal configuration shown in fig.
- **Path cost:** each step costs 1, so the path cost is just the length of the path.

State Space Search / Problem Space Search:

The state space representation forms the basis of most of the AI methods.

- Formulate a problem as a **state space search** by showing the legal problem states, the legal operators, and the initial and goal states.
- A **state** is defined by the specification of the values of all attributes of interest in the world
- An **operator** changes one state into the other; it has a precondition which is the value of certain attributes prior to the application of the operator, and a set of effects, which are the attributes altered by the operator
- The **initial state** is where you start
- The **goal state** is the partial description of the solution

Search strategies:

Search: Searching is a step by step procedure to solve a search-problem in a given search space. A search problem can have three main factors:

Search Space: Search space represents a set of possible solutions, which a system may have. Start State: It is a state from where agent begins the search.

Goal test: It is a function which observe the current state and returns whether the goal state is achieved or not.

Properties of Search Algorithms

Which search algorithm one should use will generally depend on the problem domain. There are four important factors to consider:

1. **Completeness** – Is a solution guaranteed to be found if at least one solution exists?
2. **Optimality** – Is the solution found guaranteed to be the best (or lowest cost) solution if there exists more than one solution?
3. **Time Complexity** – The upper bound on the time required to find a solution, as a function of the complexity of the problem.
4. **Space Complexity** – The upper bound on the storage space (memory) required at any point during the search, as a function of the complexity of the problem.

Searching

Many traditional search algorithms are used in AI applications. For complex problems, the traditional algorithms are unable to find the solution within some practical time and space limits. Consequently, many special techniques are developed; using heuristic functions. The algorithms that use heuristic functions are called heuristic algorithms. Heuristic algorithms are not really intelligent; they appear to be intelligent because they achieve better performance.

Heuristic algorithms are more efficient because they take advantage of feedback from the data to direct the search path.

Uninformed search

Also called blind, exhaustive or brute-force search, uses no information about the problem to guide the search and therefore may not be very efficient.

Informed Search:

Also called heuristic or intelligent search, uses information about the problem to guide the search, usually guesses the distance to a goal state and therefore efficient, but the search may not be always possible.

Uninformed search

- Breadth-First Search (BFS)
- Depth-First Search (DFS)

Breadth-First Search (BFS) is one of the simplest and most widely used algorithms for searching a graph. It systematically explores all nodes in the graph to find a solution, starting from a given starting point (or root node) and moving outward layer by layer. The algorithm does not make assumptions about the possible location of the solution but explores all nodes equally, ensuring that it checks every possibility until it finds the target node or exhausts all options.

Key Terminology in BFS:

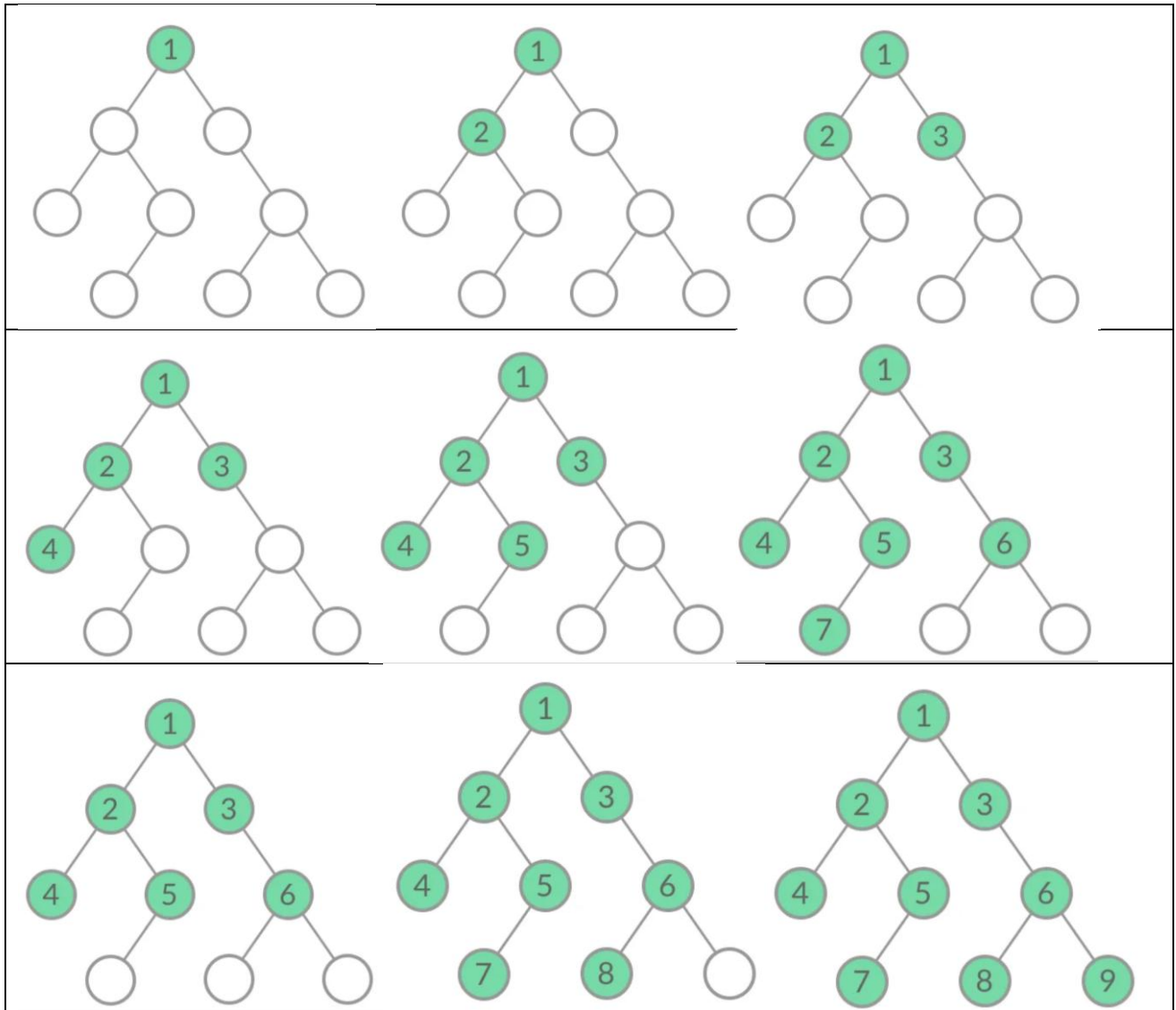
- **Graph:** A structure consisting of nodes (also called vertices) and edges connecting them. In BFS, the graph can be directed or undirected.
- **Node (Vertex):** The individual elements or points in a graph where edges meet. BFS begins at a specific node and explores outward from it.
- **Edge:** A connection between two nodes. In BFS, edges define how nodes are connected and dictate the traversal path.
- **Queue:** A First-In-First-Out (FIFO) data structure used in BFS to track nodes that need to be explored. Nodes are added to the queue when discovered and removed when explored.
- **Visited Set:** A collection of nodes that have already been explored to avoid revisiting the same node multiple times.
- **Root/Starting Node:** The node where the BFS algorithm begins. From this node, the search proceeds to explore the graph.
- **Level/Layer:** Refers to the distance from the starting node. BFS explores all nodes at the same level before moving on to nodes further away.

How BFS Works

BFS operates by exploring a graph level by level, starting from a given node and expanding outward based on distance. The detailed steps are:

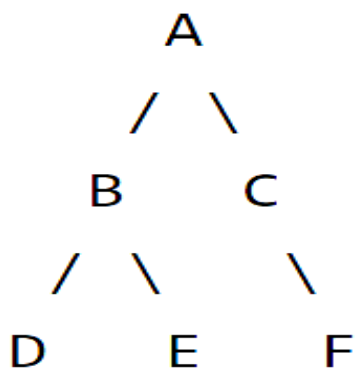
- **Mark the start node as visited** and place it in a queue.
- **Remove a node from the queue**, visit it, and perform necessary actions.
- **Add all unvisited neighboring nodes** of the current node to the queue and mark them as visited.
- **Repeat steps 2 and 3** until the queue is empty, indicating that all reachable nodes have been processed.

This process continues level by level, visiting each node's neighbors, until all nodes have been explored or the target is found. BFS is often used as the foundation for more advanced algorithms, such as Dijkstra's algorithm for finding the shortest path and Prim's algorithm for constructing minimum spanning trees.



Example Graph

Consider the following graph structure:



In this graph, nodes (also known as vertices) are represented by letters, and edges are the connections between these nodes.

BFS Traversal on the Example Graph

BFS explores the graph level by level, starting from a chosen root node (vertex A in this case). Here's how the traversal proceeds:

1. Initialization:

- Enqueue the starting node A into a queue.
- Mark A as visited to avoid revisiting it.

2. Exploration:

- **Step 1:** Dequeue A from the queue (current node) and visit it. Then enqueue its neighbors B and C, and mark them as visited.
- **Step 2:** Dequeue B and visit it. Then enqueue its neighbors D and E, marking them as visited.
- **Step 3:** Dequeue C and visit it. Then enqueue its neighbor F, and mark F as visited.
- **Step 4:** Dequeue D and visit it. Since D has no neighbors, move to the next node.
- **Step 5:** Dequeue E and visit it. Similarly, E has no neighbors.
- **Step 6:** Dequeue F and visit it. F has no neighbors.

Final BFS Traversal Order:

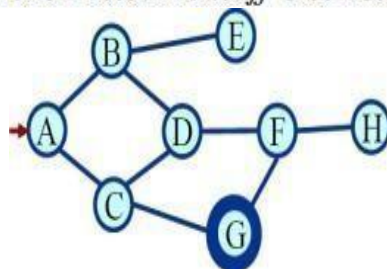
The nodes are visited in the following order:
A -> B -> C -> D -> E -> F

Depth- First- Search.

We may sometimes search the goal along the largest depth of the tree, and move up only when further traversal along the depth is not possible. We then attempt to find alternative offspring of the parent of the node (state) last visited. If we visit the nodes of a tree using the above principles to search the goal, the traversal made is called depth first traversal and consequently the search strategy is called *depth first search*.

function DEPTH-LIMITED-SEARCH(*problem*, *limit*) **returns** a solution, or failure/cutoff
return RECURSIVE-DLS(MAKE-NODE(*problem*.INITIAL-STATE), *problem*, *limit*)

function RECURSIVE-DLS(*node*, *problem*, *limit*) **returns** a solution, or failure/cutoff
if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)
else if *limit* = 0 **then return** *cutoff*
else
 cutoff_occurred? $\leftarrow$ false
 for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**
 child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)
 result $\leftarrow$ RECURSIVE-DLS(*child*, *problem*, *limit* - 1)
 if *result* = *cutoff* **then** *cutoff_occurred?* $\leftarrow$ true
 else if *result* $\neq$ failure **then return** *result*
 if *cutoff_occurred?* **then return** *cutoff* **else return** failure



DFS illustrated:

A State Space Graph Step 1: Initially fringe contains only the node for A.

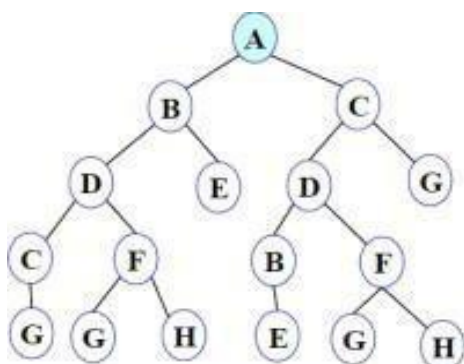


Figure 1

FRINGE: A

Step 2: A is removed from fringe. A is expanded and its children B and C are put in front of fringe.

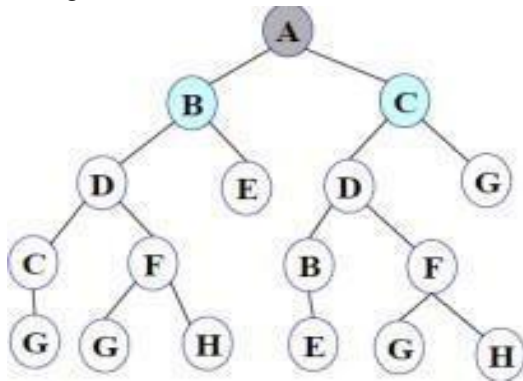


Figure 2

FRINGE: B C

Step 3: Node B is removed from fringe, and its children D and E are pushed in front of fringe.

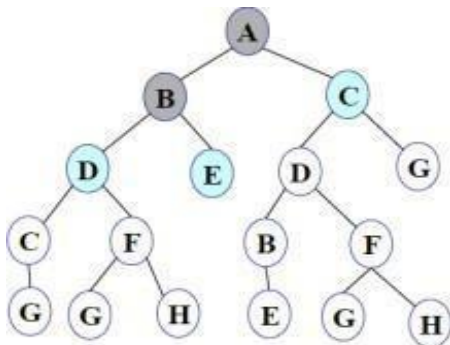


Figure 3

FRINGE: D E

Step 4: Node D is removed from fringe. C and F are pushed in front of fringe.

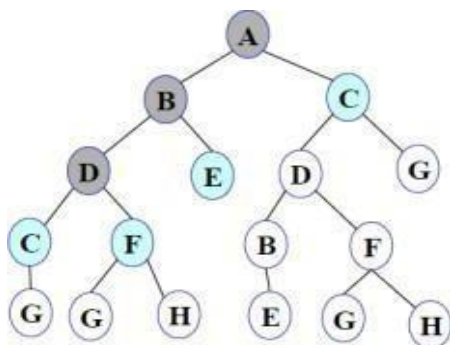


Figure 4

FRINGE: C F E C

Step 5: Node C is removed from fringe. Its child G is pushed in front of fringe.

Figure 5

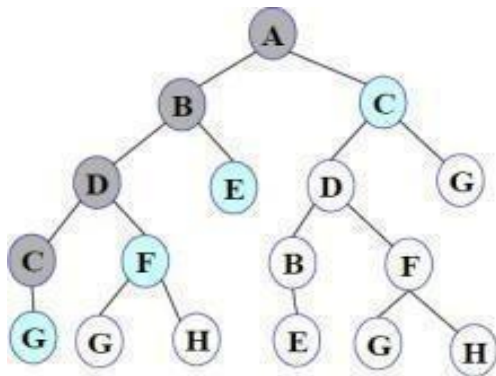


Figure 5

FRINGE: G F E C

Step 6: Node G is expanded and found to be a goal node.

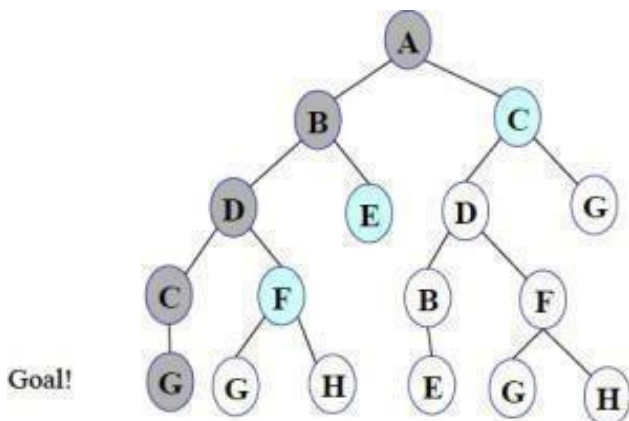


Figure 6

FRINGE: G F E C

The solution path A-B-D-C-G is returned and the algorithm terminates.

Depth first search

1. takes exponential time.
2. If N is the maximum depth of a node in the search space, in the worst case the algorithm will take time $O(b^N)$.
3. The space taken is linear in the depth of the search tree, $O(bN)$.

Note that the time taken by the algorithm is related to the maximum depth of the search tree. If the search tree has infinite depth, the algorithm may not terminate. This can happen if the search space is infinite. It can also happen if the search space contains cycles. The latter case can be

handled by checking for cycles in the algorithm. Thus **Depth First Search is not complete.**

Iterative Deeping DFS

- The iterative deepening algorithm is a combination of DFS and BFS algorithms.
- This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.
- This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.

Advantages:

- It combines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency.

Disadvantages:

- The main drawback of IDDFS is that it repeats all the work of the previous phase.

Uniform Cost Search

Introduction

Uniform Cost Search (UCS) is an uninformed search algorithm for traversing a graph. UCS evaluates and explores nodes by considering the cost to reach the node (g). The algorithm selects the node with the lowest g value to explore next. We typically use a priority queue (frontier) to select the node with the lowest g value for expansion. It is a fundamental algorithm used in various applications such as pathfinding, robotics, route planning, and optimization problems due to its completeness, optimality, and optimal efficiency.

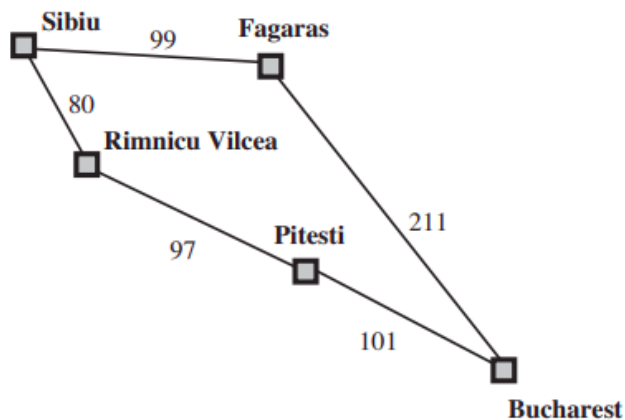
Algorithm

The UCS Search algorithm is a pathfinding algorithm used to find the shortest path from a start node to a goal node in a graph with edge weights, taking into account the actual cost from the start node to the node. The cost function $g(n)$ gives the actual cost to reach node n from the start node and is what the priority queue is ordered by.

- STEP 1: Initialize the frontier (priority queue) with the start node 'start' and set the path cost $g(\text{start})$ to 0.
- STEP 2: Initialize an empty set "visited" to keep track of the explored nodes.
- STEP 3: While the frontier is not empty, select the node with the lowest path cost value (g), which represents the least-cost node.

- STEP 4: If the selected node is the goal node, reconstruct and return the path from the start node to the goal node.
- STEP 5: If the selected node is not the goal node, remove the node from the frontier and add it to the explored set. Expand the selected node by exploring its neighbors and update the cost and parent of each neighbor.
- STEP 6: For each neighbor, if it is not already in the visited set, calculate the tentative g value, update it if it is lower than the current g value, and add it to the frontier.
- STEP 7: Repeat steps 3-6 until the goal node is reached or the frontier is empty.

Example



The task is to navigate from Sibiu to Bucharest using a uniform-cost search. From Sibiu, the connected nodes are Rimnicu Vilcea and Fagaras, with travel costs of 80 and 99, respectively. Since Rimnicu Vilcea has the lower cost, it is explored first. Expanding Rimnicu Vilcea reveals a path to Pitesti with a cumulative cost of $80+97=177$.

Next, Fagaras, which is now the least-cost node, is explored. This expansion leads to Bucharest with a total cost of $99+211=310$. Although the goal node Bucharest has been reached, since it has not been selected for expansion, the algorithm continues, prioritizing Pitesti for expansion due to its lower cumulative cost. This results in an alternative route to Bucharest via Pitesti, with a total cost of $80+97+101=278$.

Now the algorithm checks to see if this new path is better than the old one; it is, so the old one is discarded. Finally, Bucharest is selected for expansion with the updated cost of 278, and the solution is returned.

Advantages

Optimality: UCS guaranteed finding the optimal solution in terms of cost, provided that the step costs are non-negative.

Completeness: UCS is complete, meaning it will always find a solution if one exists, as long as the step costs are greater than some small positive constant.

Flexibility: UCS can handle graphs with varying edge costs and does not require any assumptions about the structure of the graph other than non-negative edge costs. This makes it suitable for a wide range of problems.

Disadvantages

Time Complexity: UCS can have a high time complexity, especially when dealing with graphs with high branching factors or when the optimal solution is deep in the search tree. Its time complexity is very high in cases where UCS explores large trees of small steps before exploring paths involving large and perhaps useful steps.

Space Complexity: UCS requires storing all explored nodes in memory to determine the optimal path. In scenarios with large graphs or limited memory resources, this can become a significant drawback, as the memory usage increases with the size of the search space.

Sensitivity to Edge Costs: UCS assumes that the step costs between nodes are non-negative. If there are negative edge costs or if the cost of reaching a node changes dynamically during the search, UCS may not guarantee optimal results.

Inefficiency with Uniform Costs: While UCS excels in finding optimal solutions when step costs vary, it can be inefficient in scenarios where all step costs are uniform. In such cases, other search algorithms like Breadth First Search may perform better.

Lack of Heuristic Information: UCS does not utilize any heuristic information about the problem domain. While this ensures optimality, it may lead to suboptimal performance in domains where heuristic information can guide the search more efficiently, such as in informed search algorithms like A*.

Comparison:			
Algorithm	Time complexity	Space complexity	Description
Breadth - First Search (BFS)	$O(b^d)$	$O(b^d)$	Explores all nodes level by level. guarantees shortest path but can be memory-intensive.
Depth - First Search (DFS)	$O(b^m)$	$O(b \cdot m)$	Explores nodes deeply before backtracking. Memory-efficient but does not guarantee shortest path.
Uniform cost search (UCS)	$O(b^d)$	$O(b^d)$	Expands the least costs node first. guarantees least-cost path but is memory intensive.

Informed search/Heuristic search

A *heuristic* is a method that

- might not always find the best solution **but** is guaranteed to find a good solution in reasonable time. By sacrificing completeness it increases efficiency.
- Useful in solving tough problems which
 - could not be solved any other way.
 - solutions take an infinite time or very long time to compute.

Calculating Heuristic Value:

- 1. Euclidian distance- used to calculate straight line distance.
- 2. Manhattan distance-If we want to calculate vertical or horizontal distance

For ex: 8 puzzle problem

Source state

1	3	2
6	5	4
	8	7

destination state

1	2	3
4	5	6
7	8	

Then the Manhattan distance would be sum of the no of moves required to move each number from source state to destination state.

Number in 8 puzzle		2	3	4	5	6	7	8
No.of moves to reach destination		2	1	2	0	2	2	0

1. No. of misplaced tiles for 8 puzzle problem

Source state

1	3	2
6	5	4
	8	7

Destination state

1	2	3
4	5	6
7	8	

Here just calculate the number of tiles that have to be changed to reach goal state Here 1,5,8 need not be changed

2,3,4,6,7 should be changed, so the heuristic value will be 5(because 5 tiles have to be changed)

INFORMED SEARCH ALGORITHMS

So far we have talked about the uninformed search algorithms which looked through search space for all possible solutions of the problem without having any additional knowledge about search space. But informed search algorithm contains an array of knowledge such as how far we are from the goal, path cost, how to reach to goal node, etc. This knowledge help agents to explore less to the search space and find more efficiently the goal node.

The informed search algorithm is more useful for large search space. Informed search algorithm uses the idea of heuristic, so it is also called Heuristic search.

Heuristics function: Heuristic is a function which is used in Informed Search, and it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal. The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time. Heuristic function estimates how close a state is to the goal. It is represented by $h(n)$, and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.

Admissibility of the heuristic function is given as:

1. $h(n) \leq h^*(n)$

Here $h(n)$ is heuristic cost, and $h^*(n)$ is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Pure Heuristic Search:

Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value $h(n)$. It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.

On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues until a goal state is found.

In the informed search we will discuss two main algorithms which are given below:

- **Best First Search Algorithm(Greedy search)**
- **A* Search Algorithm**

1.) BEST-FIRST SEARCH ALGORITHM (GREEDY SEARCH):

Greedy best-first search algorithm always selects the path which appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms. With the help of best-first search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

$$1. \quad f(n) = g(n).$$

Where, $h(n)$ = estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the priority queue.

Best first search algorithm:

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.

- **Step 3:** Remove the node n , from the OPEN list which has the lowest value of $h(n)$, and places it in the CLOSED list.
- **Step 4:** Expand the node n , and generate the successors of node n .
- **Step 5:** Check each successor of node n , and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function $f(n)$, and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

Advantages:

- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

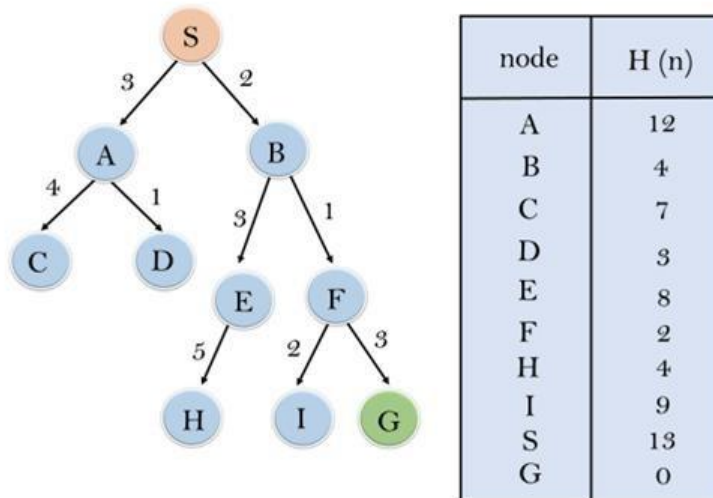
Disadvantages:

- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

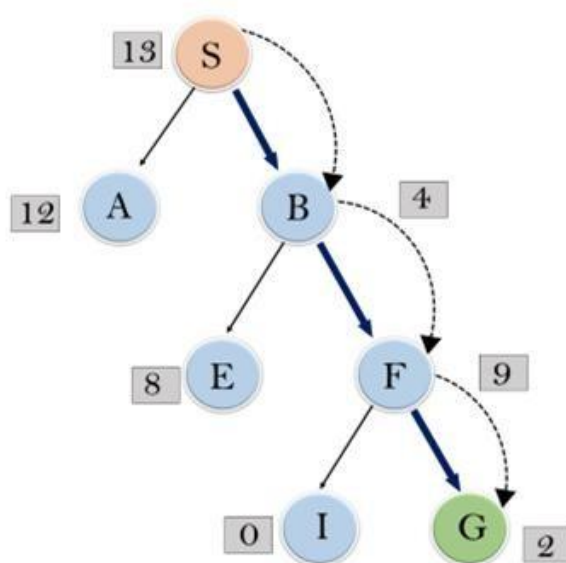
Example:

Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function $f(n)=h(n)$, which is given in the below table.

Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function $f(n)=h(n)$, which is given in the below table.



In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example.



Expand the nodes of S and put in the CLOSED list

Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open[E, F, A], Closed[S, B]

: Open [E, A], Closed [S, B, F]

Iteration 3: Open[I,G,E,A], Closed[S,B,F]

: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be: **S----> B----->F -----> G**

Time Complexity: The worst case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

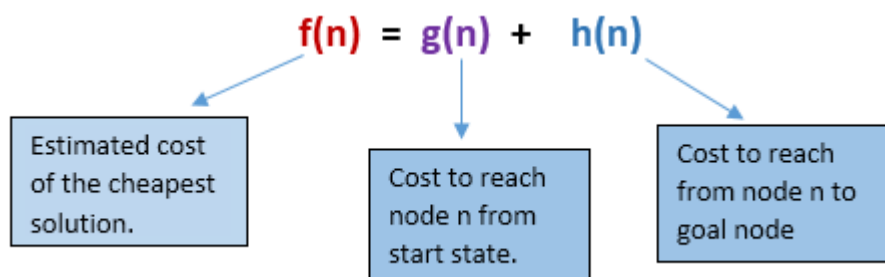
Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

2.) A* SEARCH ALGORITHM:

A* search is the most commonly known form of best-first search. It uses heuristic function $h(n)$, and cost to reach the node n from the start state $g(n)$. It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses $g(n)+h(n)$ instead of $g(n)$.

In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a **fitness number**.



At each point in the search space, only those node is expanded which have the lowest value of $f(n)$, and the algorithm terminates when the goal node is found.

Algorithm of A* search:

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function ($g+h$), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n' , check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest $g(n')$ value.

Step 6: Return to **Step 2**.

Advantages:

- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

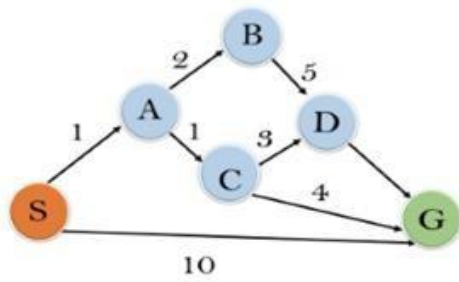
Disadvantages:

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

Example:

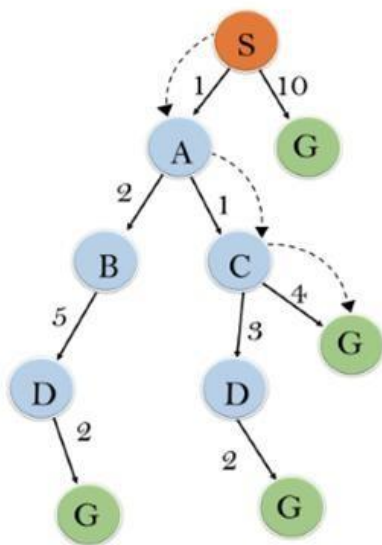
In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the $f(n)$ of each state using the formula $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.



State	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

Solution:



Initialization: $\{(S, 5)\}$

Iteration1: $\{(S \rightarrow A, 4), (S \rightarrow G, 10)\}$

Iteration2: $\{(S \rightarrow A \rightarrow C, 4), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$

Iteration3: $\{(S \rightarrow A \rightarrow C \rightarrow G, 6), (S \rightarrow A \rightarrow C \rightarrow D, 11), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$

Iteration 4 will give the final result, as **S→A→C→G** it provides the optimal path with cost 6.

Points to remember:

- A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- The efficiency of A* algorithm depends on the quality of heuristic.
- A* algorithm expands all nodes which satisfy the condition $f(n) \leq l_i$

Complete: A* algorithm is complete as long as:

- Branching factor is finite.
- Cost at every action is fixed.

Optimal: A* search algorithm is optimal if it follows below two conditions:

- **Admissible:** the first condition requires for optimality is that $h(n)$ should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- **Consistency:** Second required condition is consistency for only A* graph-search.

If the heuristic function is admissible, then A* tree search will always find the least cost path.

Time Complexity: The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d . So the time complexity is $O(b^d)$, where b is the branching factor.

Space Complexity: The space complexity of A* search algorithm is **$O(b^d)$**